From Loop to Agent

Build, connect and guard a real AI agent



IIT Agentic AI Workshop | Day 1, Session 3 | Jitesh Sureka

Today: one agent, six upgrades



Basic agent

6 min



Patterns

5 min



MCP

5 min



RAG

4 min



SDK + guardrails

5 min



70% rule

3 min

Running example: one Lost & Found assistant, upgraded a step at a time.

Our example: Lost & Found

```
{ "id": 10,
  "name": "Casio FX-82 calculator",
  "location": "Exam hall B",
  "status": "lost",
  "contact": "Reported by Mehedi" }
```

12 items in data/items.json



`search_items(query)`

find items by keyword



`get_item(id)`

read one item

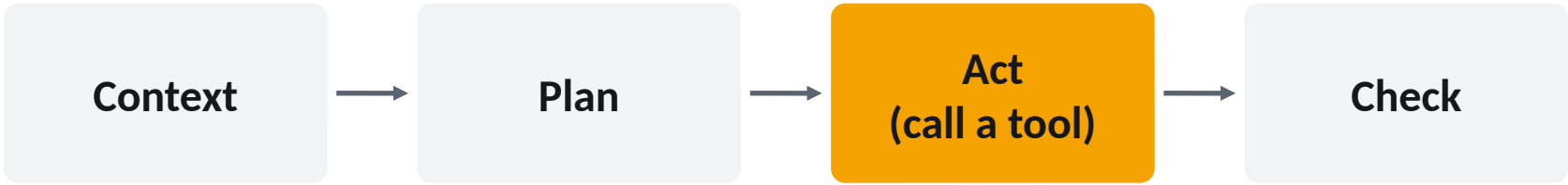


`add_item(...)`

writes to the file

Two tools read. One tool writes. Remember that for the guardrails.

Agent = model + tools + loop



repeat until the model answers, or the step cap hits (6)

The model

Reads the context. Says which tool to call and with what arguments. Never runs anything.

Your code

Runs the tool. Sends the result back. Enforces the step cap. The loop is yours.

The loop in code

```
for (let step = 1; step <= maxSteps; step++) {
  const res = await client.messages.create(
    { model, system, tools: toolDefs, messages });
  messages.push({ role: "assistant",
                  content: res.content });
  const calls = res.content
    .filter(b => b.type === "tool_use");
  if (!calls.length) return finalText(res);

  // run each tool, send the results back
  messages.push({ role: "user",
    content: calls.map(runAndWrap) });
}
```

1. Model answers, or asks for a tool
2. Our code runs the tool
3. Result goes back as the next message

LIVE DEMO `node demos/01-agent.mjs "Are there any lost calculators?"`

Three ways to run the loop



ReAct

Think, act, observe, repeat.

Best when: Open-ended tasks where the path is unknown.

Baseline



Plan first

Write a numbered plan, then follow it.

Best when: Multi-step tasks with a known shape.

+1 model call



Reflection

Draft, score it, rewrite once.

Best when: Answers where quality beats speed.

+1 model call

Do the extra calls pay off?

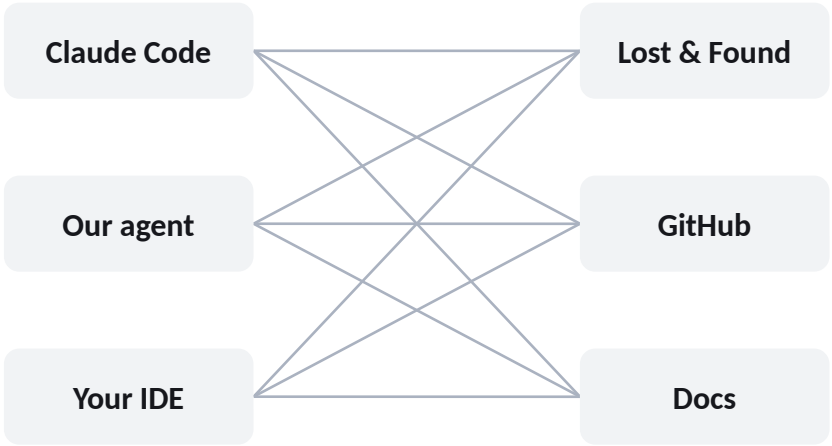
Pattern	Extra calls	Helps with	Watch out
ReAct	0	Unknown paths, recovering from tool errors	Can wander. Keep the step cap
Plan first	+1	Multi-step tasks, fewer wasted calls	A bad plan sends the loop the wrong way
Reflection	+1	Wording, completeness, format	The critic has no tools, so it cannot check facts

Numbers come from the terminal, not this slide: compare steps, tokens and correct.

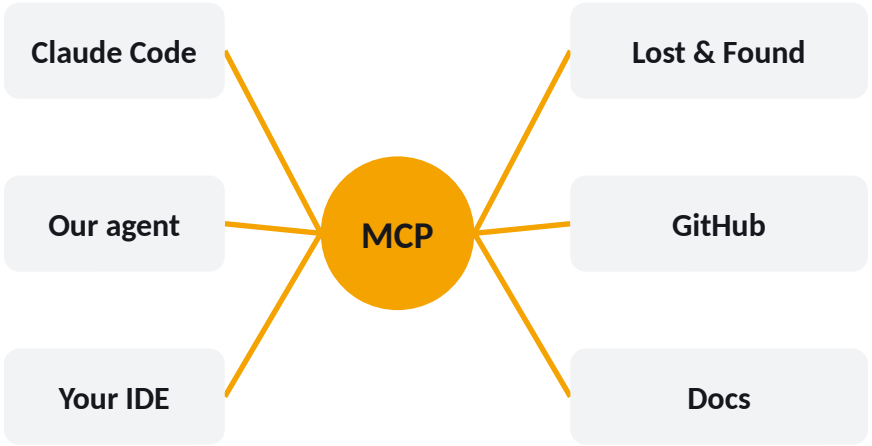
LIVE DEMO `node demos/02-patterns.mjs`

MCP: one plug for every tool

Before: N x M custom glue



After: N + M, one protocol



9 integrations become 3 + 3.

Our tool as an MCP server

```
const server = new McpServer(
  { name: "lostfound", version: "1.0.0" });

for (const name of ["search_items", "get_item"]) {
  server.registerTool(name,
    { description, inputSchema },
    async (args) => ({ content: [
      { type: "text", text: runTool(name, args) }
    ] }));
}

await server.connect(new StdioServerTransport());
```



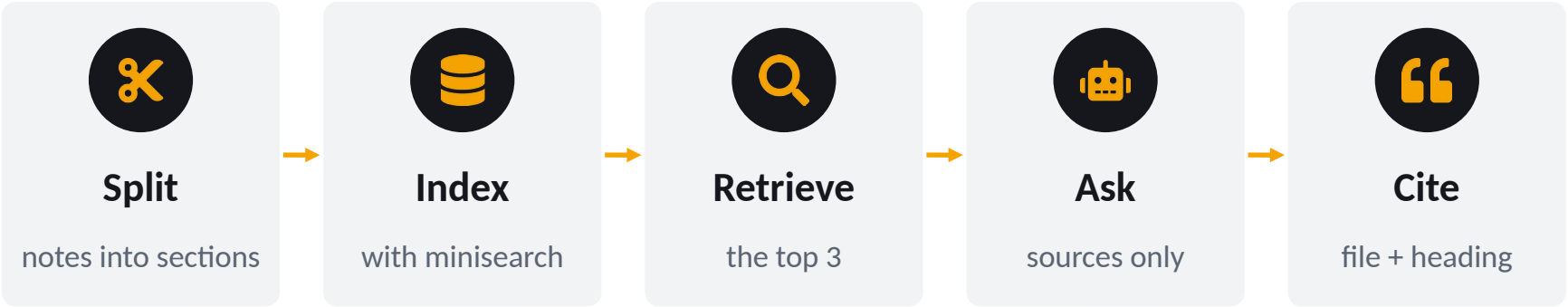
The same server works in Claude Code and in our SDK agent.



add_item is not exposed. Writes stay with a human.

LIVE DEMO `claude mcp add lostfound -- node demos/03-mcp-server.mjs`

RAG: answer from real documents



Answerable

What happens when the context window fills up?

Older content is dropped or summarized. [context-window.md > When it fills up]

Not answerable

What is the Wi-Fi password in Lab 3?

not in the notes

Cite, or say you don't know

```
// top 3 sections
const hits = retrieve(question);
// no evidence: skip the model call
if (hits.length === 0)
  return "not in the notes";

system:
  "Answer ONLY from the sources.
   Cite every claim as [file > heading].
   If the sources do not answer, reply
   exactly: not in the notes"
```

1. Cite every claim, so a human can check it.
2. No evidence means no model call: cheaper, and it cannot hallucinate.
3. Tune retrieval. Prefix search turned "Wi-Fi" into "window" until we limited it to 4+ letters.

LIVE DEMO `node demos/04-rag.mjs "What is the Wi-Fi password in Lab 3?"`

Agent SDK: delete the loop

Code you stop writing

- ❌ The message list
- ❌ The loop and the step counter
- ❌ Tool lookup and dispatch
- ❌ Error wrapping for tool results

```
query({ prompt, options: {
  mcpServers: { lostfound: {
    command: "node",
    args: ["03-mcp-server.mjs"] } },
  allowedTools: [
    "mcp__lostfound__search_items" ],
  tools: [],      // no Bash, no files
  maxTurns: 6,    // the step cap
} })
```

You still own: the system prompt, which tools exist, and the limits.

Four guardrails around the agent

1 Input

Cap length at 300 chars

`guardInput(task)`

before the model

2 Tool args

Validate every call with zod

`schema.parse(args)`

around each tool

3 Approval

Human says y/n before a write

`await confirm(...)`

before add_item

4 Output

Hide phone numbers

`redactPhones(text)`

after the model

Guardrails are code, not prompts. A prompt can be argued with.

Approval before any write

```
const r = await runAgent(task, {
  beforeTool: async (call) => {
    if (call.name === "add_item" &&
        !(await confirm("Allow this write?")))
      return "Human denied this write.";
  },
});
return redactPhones(r.text);
```

1. Denied tools return an error the model can read and recover from.
2. No terminal attached? The default is deny.
3. Phone numbers never reach the student, even if the data has them.

Three test inputs: a normal lookup, a write request, a 400-character blob.

LIVE DEMO `node demos/06-guardrails.mjs`

The 70% rule



Agents do this fast

- Scaffolding and boilerplate
- Happy-path features
- Standard patterns and wiring
- A first draft of tests

The last 30% needs an engineer

- Edge cases and failure paths
- Security and data privacy
- Fit with the real system
- Is this the right thing to build?

What a human must own



Design

Should add_item be reachable over MCP at all?



Tests

Does "not in the notes" really fire when search finds nothing?



Decisions

Who may see a student's phone number?

Every example is from today's agent. Your hands-on Step 4 finds more.

Six upgrades, one agent

1. A loop that calls tools, with a step cap
2. Patterns: ReAct, plan first, reflection
3. MCP: one plug, tools reused everywhere
4. RAG: cited answers, or "not in the notes"
5. Agent SDK: the loop is no longer yours
6. Guardrails in code, and a human on every write

Your turn: build it

7 steps. Each has a "Done when" check. Do not move on until it passes.

Short on time? Skip Step 4 and Step 6 first.
Full handout: handout/student-task.md

Setup (5 min)

```
cp -r starter lost-and-found
cd lost-and-found
npm init -y && npm pkg set type=module
npm install @anthropic-ai/sdk \
  @anthropic-ai/claude-agent-sdk \
  @modelcontextprotocol/sdk minisearch zod
export ANTHROPIC_API_KEY=<from instructor>
```

1. Model for every step: claude-haiku-5-5
2. The key is for this workshop only. Never put it in code, a commit or a chat.
3. Data: data/items.json. Notes: notes/*.md

Check: node --version prints v20 or higher.

Step 1: Basic agent (20 min)

DO THIS

1. Write 3 tools over data/items.json: search_items, get_item, add_item
2. Describe each tool to the model: name, description, input schema
3. Loop: call the model, run the tool it asks for, send the result back
4. Cap at 6 steps. Print one line per step
5. System prompt: answer only from tool results

DONE WHEN

```
node agent.mjs "Are there any lost calculators?"
```

makes 1 to 2 tool calls and names both calculators: the found one in Lab 3, and the lost one reported from Exam hall B.

Step 2: Compare patterns (20 min)

DO THIS

1. T1: a FOUND calculator, where? T2: lost Arduino kit, where to collect? T3: found items in the library, which floor?
2. Run each task as ReAct, as plan-first, and with a reflection pass
3. Plan-first: one extra call writes a numbered plan, then the loop follows it
4. Reflection: score the answer 1-10, rewrite once if below 8
5. Record steps, tokens (input + output, all calls) and correct yes/no

DONE WHEN

You have a 3x3 table of results and one sentence:

"Use ___ when ___."

Step 3: Expose it over MCP (15 min)

DO THIS

1. Create mcp-server.mjs with McpServer and StdioServerTransport
2. Register search_items and get_item. Do NOT register add_item
3. claude mcp add lostfound -- node /full/path/mcp-server.mjs
4. Ask Claude Code: "any lost calculators?"

DONE WHEN

Claude Code shows a call to `mcp__lostfound__search_items` and answers from your data.

Step 4: The 70% rule (10 min)

DO THIS

1. Open your Session 2 autonomous output (or your Step 1 agent)
2. List what is missing or weak: tests, errors, edge cases, design, security
3. Mark each item: Agent can do, or Human must own
4. Pick the one human-owned item that hurts most if wrong. One line on why

DONE WHEN

Your list has at least 6 items.

Every item is marked.

One item is circled, with a reason.

Step 5: RAG with citations (20 min)

DO THIS

1. Split every notes/*.md file at each "## " heading
2. Index sections with minisearch (heading, text). Drop stop words like "the", "of"
3. Send the top 3 sections to the model: answer only from sources, cite [file > heading]
4. No search hits? Answer "not in the notes" without calling the model

DONE WHEN

Context window question: answered, cites context-window.md.

"Wi-Fi password in Lab 3?" gets exactly:

not in the notes

Step 6: Agent SDK (15 min)

DO THIS

1. Rebuild the agent with query() from @anthropic-ai/claude-agent-sdk
2. Point mcpServers at your Step 3 server
3. Set allowedTools, maxTurns: 6 and tools: [] (no Bash, no files)
4. Set settingSources: [] so it ignores your personal settings

DONE WHEN

It answers the Step 1 question.

You can list the code you deleted: loop, message list, tool dispatch, step counter.

Step 7: Guardrails (15 min)

#	Guardrail	How
1	**Input**	Reject a task over 300 characters before calling the model
2	**Tool args**	Validate every call with zod. On failure, send the error to the model
3	**Approval**	Ask y/n before add_item writes. Anything but y means no write
4	**Output**	Replace phone numbers like 01711-000000 with [phone hidden]

DONE WHEN

A 400-character task is blocked.

"Add: found a green scarf at Room 105" asks y/n. Answer n and items.json is unchanged.

Mehedi's contact number is never shown.

Stretch goals and hand in

Stretch goals

- Break your own guardrails two ways, then fix one
- Add mark_claimed(id) behind an approval
- Ask a clarifying question when search finds nothing
- Print the cost of each run

Show the instructor

- Your Step 2 results table
- A screenshot of Claude Code calling your MCP tool
- Your Step 4 list, with the circled item